

NILE UNIVERSITY OF NIGERIA FACULTY
OF COMPUTING DEPARTMENT OF
COMPUTER SCIENCE UNDERGRADUATE
FINAL YEAR PROJECT

**Cognitive Vision-Based Hand Gesture Recognition System for
Media and Accessibility Control**



Supervisor: Mrs.

Aisha Lawal

Scientific Advisor

Author:

Adeola Damilola Kola-

Olaleye

241030151

Abuja, 15th January, 2025

Abstract

This project discusses a real-time cognitive vision system for gesture based media control by leveraging webcam based hand and finger tracking. The goal of the system is enhancing access, and interaction for those with physical disabilities by providing touch-free control to media actions like: volume adjustments, screen brightness control, play and pause. By utilizing computer vision and low-power machine learning models, the system decodes finger landmarks from a conventional webcam in real time for high accuracy continuous gesture recognition with low latency. Continuous scale-volumes are associated with (e.g.) the distance between thumb and index-finger, conveying properties like size or amount of an item, and finite control-expression templates corresponding to postures on the hand. The system was planned to be able to perform steadily and in realtime at rates of 30-60 FPS, and remain robust with performance reduction as a result of different users and the variation in lighting conditions. This study contributes to the area of cognitiveHCI by offering an inexpensive interactive camera-based assistive technique, which goes beyond normal conventional input interactions. This project will display the combination of computer vision and cognitive modeling.[1].

Chapter 1

Introduction

1.1 Background to the Study

The evolution of Human-Computer Interaction (HCI) has been defined by a continuous effort to bridge the gap between human cognition and computer execution. From the text-based command-line interface (CLI) to the visual WIMP (Windows, Icons, Menus, Pointer) graphical user interface (GUI), mediated by electromechanical pointers like the mouse and keyboard, the physical relationship between the user and the machine is maintained and reinforced [2]. Though ubiquitous today, this reliance on a physical input device limits the extent to which the interface can exist anywhere or everywhere in the physical world. The WIMP paradigm, while the de facto standard of computer use today, is becoming more incongruous as digital interfaces enter increasingly non-traditional spaces, such as medical environments and home entertainment spaces, where it is more difficult and undesirable for the user to touch the interface directly. The Natural User Interface is a paradigm that has emerged to address this problem.

Rather than relying on a physical intermediary device, NUIs use intuitive human communication modalities, such as voice, gaze, and gestures, to communicate with the computer, rendering the interaction device nearly invisible, thus allowing the user to concentrate on the task and the content itself [3]. This research is focused on gestures as the primary modalities for communication. Specifically, human hand movement offers a highly expressive communication modality that allows for a large, high-bandwidth number of possible interactions with the computer. However, it is computationally demanding to determine the hand pose with any degree of accuracy using computer vision and to perform the necessary analysis to infer the user's intent.

Furthermore, a significant portion of the previous work has relied on depth sensors or complex neural network models that can only be trained on high-end graphical processing units (GPU) to achieve any degree of performance. While these solutions are accurate, their dependence on expensive specialized hardware, such as LiDAR, and their requirement for specialized accelerators such as GPUs limit their applicability to the

mainstream consumer electronic devices [4]. In recent years, however, there has been a marked shift in computer vision, due to the creation of lightweight machine learning models that can execute in real-time. It is now possible for models to be trained on consumer-grade Central Processing Unit's (CPU) and to achieve real-time inference on these systems [1].

In this particular project, we are focusing on the Google MediaPipe solution for gesture recognition. MediaPipe is an efficient and reliable hand-pose estimation system that can detect the position of the hand in the input image and determine the location of landmarks and pose of the hand. Unlike previous methods, which require a sliding window for each frame to calculate the location of landmarks, MediaPipe first detects the hand and, using the position as an anchor, calculates the location of the landmarks to create a skeleton for the hand [5]. This project is focused on demonstrating the viability of this hand-pose estimation system by creating a software application that can utilize the position of the hand to control variables, such as volume level or brightness of the system display, thus eliminating the need for physical input devices.

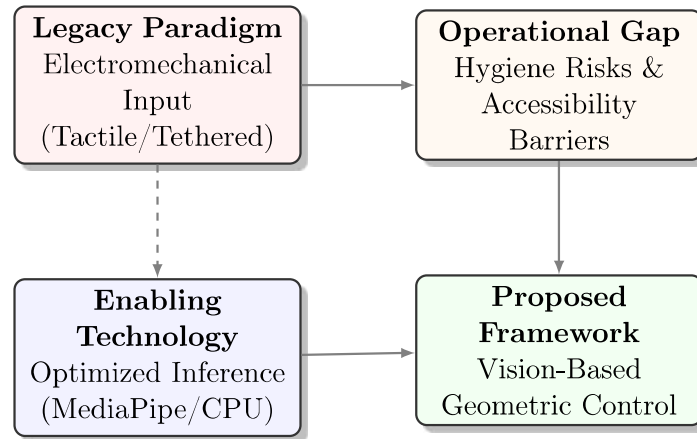


Figure 1.1: Conceptual framework illustrating the technological transition from tactile inputs to vision-based interaction.

1.2 Statement of the Problem

Despite the ubiquity of computer-based applications and their widespread use, it is difficult to operate and interface with these applications efficiently and in certain cases even possible in various circumstances. Although there exist several different applications and operating systems, there is still a reliance on a physical input device such as a keyboard, touchscreen or mouse. This reliance is not conducive or even practical in certain situations, specifically when the computer needs to operate in a variety of circumstances; the most prevalent being the need for a clean, hygienic

interface or to operate when there is physical distance between the user and the application [6].

Furthermore, certain computer users have limited motor control and fine motor control required for the operation of physical devices, as well as certain individuals that are at risk for motor disorders, as it is not possible for them to use a keyboard, mouse or touchscreen [7]. Finally, though alternative methods of interface may exist, they are either not accurate to emulate the precision of physical devices such as a keyboard or mouse, or have their own shortcomings, such as voice control, due to the possibility of interference from the surrounding environment, the lack of privacy or the limitations of the physical device, specifically the binary nature of the output [8]. Furthermore, no framework exists today that maps the Euclidean geometry of a hand landmark position to a computer system efficiently without the need for specialized hardware accelerators.

1.3 Aim and Objectives

1.3.1 Aim

Designing, development and evaluation of an HCI system that allows for real-time manipulation of Computer system parameters, Volume, Brightness and Media Navigation, using bi-manual gesture recognition.

1.3.2 Objectives

To achieve this aim, the following objectives has been set:

- i. Design of system: To design the Cognitive Vision-Based Hand Gesture system.
- ii. Implementation of system: To define an algorithm that calculates the Euclidean distance between nodal key points to determine gesture intensity.
- iii. Performance Benchmarking: To validate the system performance of the application by quantifying the FPS (frames per second), latency (ms) and system resources (CPU) usage.

1.4 Research Questions

- i. To what degree is it possible to control physical computer devices using a hand gesture recognition framework?

- ii. What are the limitations, if any, for an algorithm designed to use a hand gesture recognition framework to control a physical device in real-time?
- iii. Is it possible to differentiate between noise (jitter), such as hand tremors, and a control signal? [9]

1.5 Scope and Limitations

1.5.1 Scope of the Study

This study is limited to a software project that uses a library of Python programs. The scope includes the following:

- i. Input source: The webcam is a USB Video Class device with a minimum resolution of 640×480 pixels.
- ii. Core functionality: The system is powered by Google MediaPipe for hand landmark detection and NumPy for performing vector calculations.
- iii. Interaction interface: The project uses the host system controls to change the sound volume and the brightness of the display.
- iv. Development environment: It uses a Windows computer that is 64-bit and the project assumes a typical indoor room has enough lighting.

1.5.2 Limitations

- i. Lighting conditions: Since this is a vision based project, the performance of the project would be affected in a very dimly lit room since it may cause poor signal detection.
- ii. Hand occlusion: As the project relies on a video-based input source, occlusion or blockage of the user's hands will lead to a failed project response.
- iii. Single user tracking: The current algorithm is optimized for a single active user; multi-user conflict resolution is beyond the scope of this iteration.

1.6 Significance of the Study

The results of this project will help the following:

- i. Human Computer Interface: The results of this project can lead to a better understanding of how geometric regression can be used instead of neural network classification.
- ii. Assistive Technology: The results of this project could be used in assistive technologies to assist people with disabilities by removing the need to physically press buttons like in other computer interface projects [10].
- iii. Hygiene Computer Interface: The results of this project can be used to make computer interfaces in public areas more hygienic since this project doesn't require touching the computer.

1.7 Assumptions of the Study

- i. The room has sufficient lighting so that the webcam sensor could detect the skin color of the user's hand.
- ii. The user has the ability to use at least one or both of his or her hands, depending on the mode of operation.
- iii. The computer can handle the AVX2 instruction sets required by the TensorFlow library used by MediaPipe.

1.8 Project Feasibility

- i. Technical Feasibility: The current state of the TensorFlow library allows the MediaPipe system to run on modern CPUs without a dedicated GPU [1]. Preliminary prototyping confirms that geometric logic executes in $O(1)$ time complexity relative to the frame buffer.
- ii. Economic Feasibility: The project uses only open-source tools. There are no financial costs in the creation of the software or acquisition of the tools and hardware needed.
- iii. Operational Feasibility: Users will only have to learn simple hand movements and will therefore not experience a steep learning curve to learn how to use the software.

1.9 Terms and Definitions

- i. Computer Vision (CV): The area of study that tries to allow computer systems to pull high-quality information from images, using machine learning, deep learning, artificial intelligence or some other algorithm.
- ii. MediaPipe: A multi-modal applied machine learning pipeline to create multimodal experiences. MediaPipe streams data from a variety of inputs such as video and audio and processes them in real-time [1].
- iii. Landmark: The 21 (x, y, z) coordinates predicted by the MediaPipe hand model which are the skeletal joints of the users hand.
- iv. Euclidean Distance: Distance between any two points in Euclidean space or 3d space as determined by Pythagoras Theorem. This distance is the calculation for the magnitude of the hand gesture.
- v. Latency: The amount of time that passes between when a user initiates an action and when an action takes affect by software.
- vi. Region of Interest (ROI): The part of the image that will be analyzed, which makes the analysis take less time.
- vii. Jitter: Jitter is the sudden small movement seen in the output of the system where the computer attempts to locate the users hand when the user is stationary, or when the light is not bright enough. This is why filters will be added to the project so that the audio volume does not move independently if the user is stationary or if the light is not bright enough.
- viii. Frames per second (fps): The frequency at which sequential images, aka frames, appear on a display. In order for a gesture system to feel like a real-time system it will need to be 20-30 FPS to maintain the effect of motion.



Figure 1.2: Keypoint localization of 21 hand-knuckle coordinates within the hand regions [1].

Chapter 2

Literature Review

2.1 Conceptual Review

The section covers the theories and concepts that inform the development of computer vision based Natural User Interfaces (NUI) that goes beyond "what" is being developed to focus on "why?" We explore cognitive load theory, accessibility theory, and geometric mathematics as it pertains to computer vision.

2.1.1 Paradigm shifts in Human Computer Interaction (HCI)

The field of study Human Computer Interaction (HCI) focuses on the relationship between humans (users) and computers. The history of HCI is a pursuit in decreasing the Gulf of Execution. The Gulf of Execution is defined as the distance between the users objective, to execute an action, and the chain of operations necessary to complete the objective in a given system.

- i. CLI (Command Line Interface): Users are required to memorize the specific syntax of commands in order to make the command work. The CLI presents a steep learning curve that acts as a barrier of entry for non-computer literate users.
- ii. GUI (Graphical User Interface): Utilizing a mouse, keyboard, monitor, this interface introduces what WIMP stands for: Windows, Icons, Menus, Pointer. While there is a reduced learning curve compared to a CLI and there is an introduction of visual metaphors it is still clear that there are barriers to keep the user separated from the system itself. The mouse and keyboard are peripheral devices and separate the user from direct interaction with the system.
- iii. NUI (Natural User Interfaces): Natural User Interfaces are the paradigm of interface that is targeted in the current project. Natural User Interfaces seek to remove the need for external peripheral devices and allow the user to interact organically with the system in their natural environment. Hand tracking provides a high bandwidth

interaction channel, however this process has historically had high computational demands for use in consumer based hardware.

2.1.2 Cognitive Load and Motor Control Theory

One of the driving factors behind this project is the concept of reducing cognitive load. With traditional input devices the user must bridge the cognitive load of interpreting movement in one physical plane (your hand moving on the mouse on a horizontal desktop plane) to that of a different plane (the cursor on the monitor in the vertical plane). Our solution takes advantage of proprioception to bridge that load gap. We utilize embodied interaction so that one physical action (a pinch) directly maps to one digital action (turning down the volume).

2.1.3 Accessibility and Digital Inclusion

By lowering cognitive burden, this concept of computing also tackles the problem of digital accessibility. Generally, hardware-based interfaces rely on having a minimum amount of fine motor control and manual dexterity.

Motor impairments: If you have carpal tunnel, arthritis, or cerebral palsy, a repetitive and precise interaction method like a mouse might be a bit painful or just simply not possible for you to do at times.

Touchless Concept: We are taking the concept of universal design and lessening the need for physical effort, because we are mapping gross movements in space to small changes in volume or screen resolution, thereby making computing accessible for those who cannot physically handle a mouse or keyboard.

2.1.4 Geometric Mapping in Computer Vision

The change in the way we understand this particular level of computer vision is the change from classification (Is this a hand?) to regression (Where is this hand in relation to itself?). The geometric mapping formula mainly used is the Euclidean distance. It takes two points in space (two fingers) and applies the distance equation:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

This helps us translate the real 3D world (fingers) to 1 dimensional scalar values (0 to 100 percent) used by a physical audio fader or the volume on a digital screen.

2.2 Empirical Review and Related Works

This review examines models for gesture recognition, as they have been written about in the research literature. The aim of this work is to justify our proposal through a review of previous systems and their successors with specific citation.

2.2.1 Legacy Method: Color Based Segmentation (HSV)

There are early examples of gesture manipulation done via segmentation of the hand color. **Cite:** [11] describes an HSV color space hand segmentation control scheme for presentations via gestures.

Description: The authors generate binary mask images where a hand has its skin color pixels distinguished from its background. Next, the authors assume the hand is contained in the largest contour of a binary image.

Evaluation: This technique is computationally low cost but very nonrobust. Their results showed that they were able to differentiate the background from the hand in easy situations, but in more complex situations, like using backgrounds that resemble skin tone, or changing light conditions, their algorithm would fail to work. Furthermore, the hand model they created doesn't have any semantics, so it cannot be easily expanded to continuous (or fine-grained) gestures like what we use in this thesis.

2.2.2 Legacy Method: Convex Hull and Defect Analysis

Others have also built upon segmentation and have developed approaches that use convexity analysis to count fingers.

- i. Reference: [12] present a technique that combines background subtraction with convexity analysis to identify hand pose.
- ii. The system: the authors draw a convex hull (polygon) to enclose the hand (a set of skin pixels). This hull is typically used in combination with defect detection in order to calculate the number of open fingers by counting the number of defects (gaps between two convex hull points).
- iii. Critique: This technique, like the HSV-based system, is restricted to discrete pose recognition (i.e., five fingers out, or one finger, or a fist). It cannot measure a continuous distance between two feature points (e.g. distance between the tip of the thumb and the tip of the index finger) and hence could not be used for continuous volume scrubbing or similar functionality.

2.2.3 Legacy Method: Haar Cascade Classifiers

Many researchers have relied upon other well-established computer vision approaches to detect hand poses prior to the widespread use of deep learning.

- i. Reference: [13] investigated the use of Haar Cascade classifiers for sign language gesture recognition.
- ii. The system: This classifier utilizes a cascade of increasingly complex binary tests based on simple rectangular features. It has been trained on thousands of example images of both a human hand (positive samples) and of everything else (negative samples).
- iii. Critique: Like the HSV-based and Convex Hull methods, the classifier is relatively rigid: it does not learn to track a hand as a set of feature points, but simply recognizes the shape of the hand and nothing more. Moreover, the classifier is not very flexible to change of pose or size; even minor changes to the hand's shape or orientation can lead to a failure of the hand detector.

2.2.4 Hardware Dependent Method: Depth Sensors

High-accuracy gesture recognition has been achieved in previous literature, but these systems are not practical due to the hardware that they rely on.

- i. Reference: [14] utilize the Microsoft Kinect sensor and the Leap Motion Controller to achieve high accuracy for gesture recognition.
- ii. The system: These hardware devices use Infrared (IR) structured light or Time of Flight (ToF) sensors to generate a dense 3D point cloud of a human hand.
- iii. Critique: While these sensors are indeed capable of performing high accuracy gesture recognition, their reliance on specialized hardware is not consistent with the project goal of having a system that is accessible to everyone. Both the Microsoft Kinect and the Leap Motion Controllers require additional expensive, proprietary hardware to work with.

2.2.5 Deep Learning Method: OpenPose Framework

Prior to MediaPipe, CMU's OpenPose held the standard for multi-person skeleton tracking.

- i. Cited Work: [15] proposed OpenPose, a network architecture based on Part Affinity Fields (PAFs) for real-time detection of human, hand, and face keypoints.

- ii. Methodology: OpenPose employs a Bottom Up approach. It scans the whole image to detect all individual keypoints, producing heatmaps for each point, before linking them together to form complete poses.
- iii. Critique: This approach demands significant computational power. Typical performance on a CPU alone is less than 5 FPS, making it unsuitable for a real-time control surface where latency causes disorienting delay in the visual-auditory feedback loop.

2.2.6 Latest MediaPipe implementations (virtual mouse)

At present, very few applications have utilized MediaPipe to provide fundamental control of user interface. Related Work [16] presented a virtual mouse using MediaPipe Hands to perform AI based hand tracking. This work mapped the user's index finger tip to the cursor position. While demonstrating that MediaPipe has a demonstrated capacity for tracking mouse cursors, this work did not include a Bi Manual architecture which separated media scrubbing from volume control. The study also did not apply Euclidean distance as a way to measure analog inputs.

2.3 Review of Tools and Frameworks

2.3.1 OpenCV (Open Source Computer Vision Library)

This project makes use of OpenCV mainly to acquire video frames and present them.

Preprocessing video stream: Frames are acquired via `cv2.VideoCapture` then converted from BGR to RGB prior to passing to MediaPipe.

Visualization: OpenCV is also applied to draw the skeleton on frame so the user has feedback that the system is tracking their hands.

2.3.2 MediaPipe Hands

This project will utilize Google's MediaPipe Hands. MediaPipe uses a Top Down methodology to achieve real time detection on CPU only machines.

BlazePalm Detector: This is an SSD model which is applied to the first full frame, for detection of a bounding box for the detected hand. Specifically the BlazePalm model is detecting for the human palm, not the fingers, since the palm is more of a rigid body that is more simple to model compared to a finger which would need more complex analysis. This runs on the first frame or if the tracker loses tracking to conserve processing power [1].

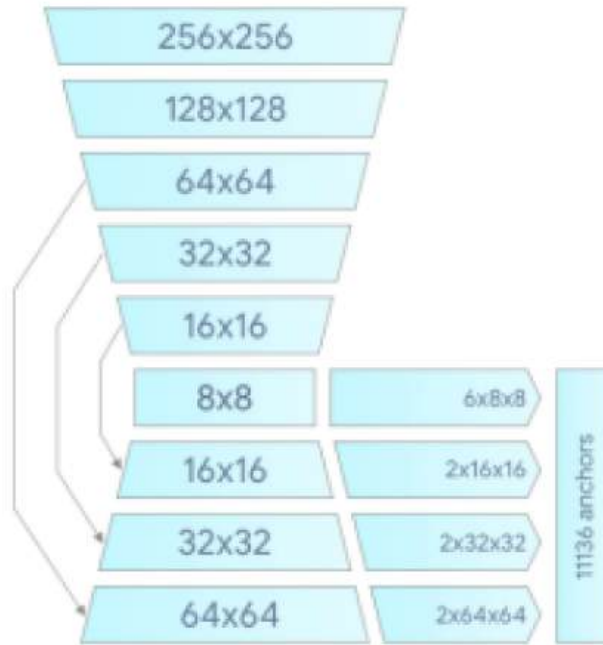


Figure 2.1: A high level view of the BlazePalm detection architecture, showing the SSD back end used to regress the bounding box [1].

Hand Landmark Model: Once a bounding box of the palm is found, the frame is cropped to a Region of Interest (ROI). Phase two utilizes a regression to detect 21 key landmarks inside the ROI and return 2.5D coordinates for each landmark point, including the x, y and z.

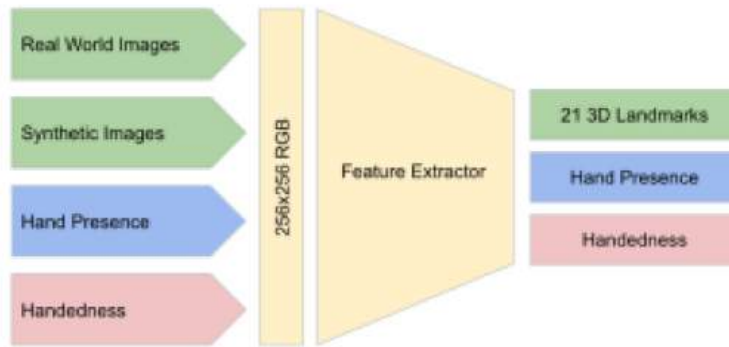


Figure 2.2: 21 hand landmark topological structure of MediaPipe used to calculate Euclidean distance for all landmarks [1].

2.3.3 NumPy and Actuation Libraries

NumPy: Used to vectorize and perform `np.interp` on physical readings, such as distance between the middle finger and index finger, into digital signals for the application to act on.

PyCaw: An API wrapper for Windows Audio Session API (WASAPI) which will allow

the application to control master channel volume levels.

Screen Brightness Control (SBC): Is implemented via the monitor's Display Data Channel/Command Interface (DDC/CI) to control monitor LED backlights.

PyAutoGUI: Temporal control implementation for a user defined period. This allows the project to send keyboard commands to the operating system, such as triggering a media play or pause button.

2.4 Identified Gaps and Comparative Analysis

After reviewing the relevant literature, we were able to identify several major gaps. Most of the prior work that came out in the 2010s was not accurate enough; current technologies such as OpenPose do not perform well enough, and even more recent technology like MediaPipe does not provide a solution to many of the accessibility concerns, such as brightness control and media scrubbing. **No Brightness Control:** There is currently very little research into DDC/CI which would allow for brightness control of photosensitive individuals to be implemented.

No Media Scrubbing: There is very little media control that allows linear video scrubbing, which should ideally be achievable via a simple horizontal swipe motion between a pinched thumb and index finger.

No Temporal Control: Many Open Source projects that provide control functions do not implement a temporal filter, meaning small involuntary or unintended movements that trigger a gesture will still trigger the associated function (e.g. small unintentional movements triggering volume controls). Instead, time gated input is used, requiring double taps.

Chapter 3

Methodology and System Analysis

3.1 Introduction

This chapter addresses the development and implementation of the proposed software application, describing the method employed in this work. We explain the system architecture and the design of each algorithm used to meet the goals of Chapter 1. Also discussed is the SDLC, requirements, as well as a derivation for the formula used for each form of control. Lastly, the Unified Modeling Language (UML) used in the modeling of each hand configuration is illustrated.

3.2 Project Approach: Waterfall

The Waterfall model is the chosen Software Development Life Cycle (SDLC) for the project. This approach is deemed more suitable for our project's needs since it can be more organized, planned and controlled when an information system software is in development [17]. In contrast to the Agile methodology which is a cyclic model, Waterfall model is a linear model. We decided on using the waterfall model based on the goal of the project as well as the timeline of the final year project in college. The project will not go through any iterations from Requirement Analysis to Maintenance & Optimization in a linear development approach. Our project goal is well-defined and that is Gesture to Action.

A typical project's Software Development Life Cycle consists of the following:

- i. Requirement Analysis and Feasibility Study: This section states the scope of the project which is what the software will be able to accomplish which is Accessibility Controls. Specifically this project will only perform Volume Control, Brightness Control and Media Scrub. The feasibility study showed that our choice of using MediaPipe which is a framework that can detect hands on CPU was a feasible software implementation [1].
- ii. System Design and Architecture: Before we begin coding, the architecture is first designed. The design follows a modular architecture with one layer for computer

vision which we will call Perception and one layer for system actions which we will call Action. We include a Data Flow Diagram which will describe where does data come from from our source and how data will be transformed to perform the necessary action. We also design the data flow diagram for the Right Hand system controls and the Left Hand Media Control.

- iii. Implementation: The process of coding the program with a specific programming language. For our project, we code in Python. We will divide coding into units for each module in our architecture. For example, one unit is to construct a HandDetector that will encapsulate the functions from MediaPipe. The next unit we will code is the mathematical calculation to define the geometric formulas. The final unit is to integrate our system libraries with our program.
- iv. Testing and Verification: During the course of the development cycle, the project must go through a phase of testing. We used a White Box testing strategy for our project. One aspect of this testing is to measure the latency which means the amount of time between a gesture from the user and our program response to said gesture.
- v. Maintenance and Optimization: During this step, we will refine the program and further optimize the code. Removed unnecessary operations in the for loop and adjusted the smoothing/interpolation on the signals (linear interpolation), the UI bars now move smoothly instead of stuttering back and forth.

3.3 Requirements Specification

Requirements specification is the list of the application requirements. This falls into two categories:

- i. Functional Requirements: What the app does (the functions).
- ii. Non-Functional Requirements: How good the app is at what it does (the quality attributes).

3.3.1 Functional Requirements

- i. **FR1: Video Acquisition and Pre-processing**
 - a. Set Webcam 0 as the default camera and grab frames $\geq 640 \times 480$ pixels.
 - b. Set the default color space for the webcam to RGB.
- ii. **FR2: Bi-Manual Hand Tracking**
 - a. Detect and track multiple hands and assign proper labeling (e.g. "Left" and

"Right") via MediaPipe's handedness output for each detected hand.

b. Extract 21 2.5D landmarks for each detected hand [5].

iii. **FR3: Right Hand System Control**

a. Use Right Hand as system controls.

b. Use a distance metric between finger landmarks to calculate the volume and brightness system control:

i. Volume is the distance between Thumb (4) and Index Finger (8).

ii. Brightness is the distance between Thumb (4) and Middle Finger (12).

c. Use Double Taps (temporal gesture between Index Finger [8] and Thumb [4]) to control play/pause status.

iv. **FR4: Left Hand Media Scrubbing**

a. Use your left hand to scrub through media content.

b. Once the left index finger and thumb are close to each other (entering an Active State), moving the hand along the x-axis (Δx) will seek the media, moving in one direction to Rewind and the other to Fast Forward.

v. **FR5: Signal Mapping and Actuation**

a. Perform a simple linear mapping of pixels to the available controls.

b. Use the right hand to pinch with a gap of 50 to 200 pixels to change the volume levels (65 dB to 0 dB) or change the brightness (0% to 100%).

c. The left hand's position in the video space from 0 to 640 pixels (on the x-axis) will be used to scrub through media using the timeline control.

vi. **FR6: Visual Feedback Interface**

a. Render a hand skeleton over the video frame to improve usability.

b. Render Volume/Brightness indicators and Scrubbing progress bar over the video frame in real-time.

3.3.2 Non-Functional Requirements

i. **NFR1: Latency and Throughput**

a. The pipeline latency of the system must be less than 50 ms.

b. The framerate must remain >20 frames per second (FPS) (for 2 hands tracking) to ensure the feedback loop remains within acceptable cognitive levels.

ii. **NFR2: Robustness and Error Handling**

a. Handover (Crossing/Occlusion): If the hands cross or occlude one another, the logic layer must choose the hand with the highest confidence, ignore the other hand,

and prevent conflicting control inputs.

b. Double Occlusion: If both hands become fully occluded, the logic layer must use only the relative hand detection confidence.

iii. **NFR3: Privacy and Data Locality**

a. Processing always takes place on the edge (enabled by default) [18]; there is no logging or transmission of data.

iv. **NFR4: System Resource Efficiency**

a. Must not hog the CPU in order to play back the media smoothly during scrubbing.

3.4 System Architecture

We designed the system as a Linear Pipeline with a Branched Logic Layer so that we could use 2 separate images as inputs, one for each hand. We ended up having a total of 5 layers:

- i. Acquisition Layer: Initialised OpenCV and setup hardware to read stream from a webcam.
- ii. Pre-processing Layer: Performed BGR-to-RGB conversion on the image.
- iii. Inference Layer: Constructed MediaPipe graph and detected landmarks and labels for each hand to identify left hand and right hand.
- iv. Logic Layer (Branched):
 - A. Right Hand:** Calculated the distance between two landmarks, normalised the result from 0 to 1 range and used it for changing Volume or Brightness.
 - B. Left Hand:** Calculated the Delta of the Centroid X-coordinate between two frames and used it for Scrubbing.
- v. Actuation Layer: Actuated upon the signals processed by the Logic Layer by issuing WASAPI and DDC/CI API calls at the Operating System level and synthesising keyboard commands from them.

3.5 Detailed Design and Modeling

3.5.1 Use Case Modeling

As shown in the Use Case Diagram (Figure 3.2), the two hands are assigned control of System State and Volume for the right hand, and Media Timeline functions for the left.

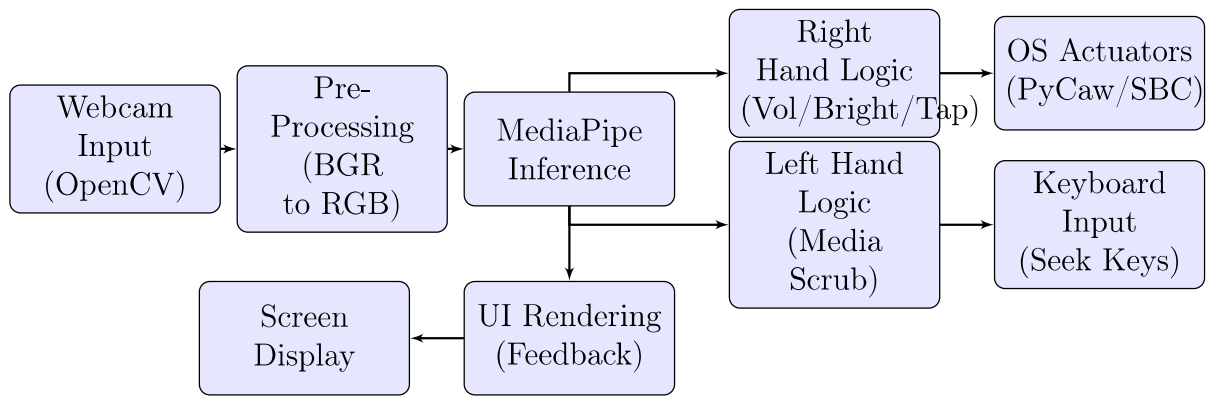


Figure 3.1: System Architecture Pipeline with Bi-Manual Logic Branching

- **Right Hand:** Controls System Params (Volume/Brightness) and System State (Play/Pause).
- **Left Hand:** Controls Media Timeline (Scrubbing).

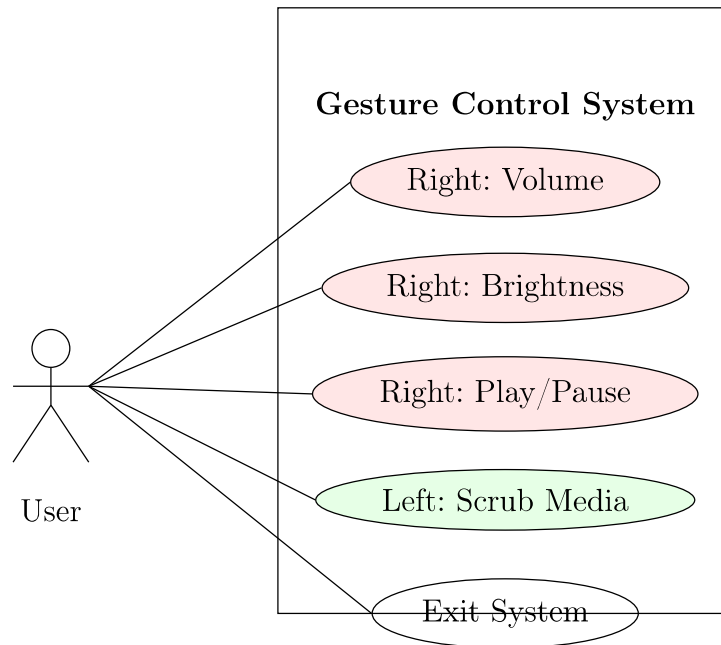


Figure 3.2: UML Use Case Diagram depicting Bi-Manual Interactions

3.5.2 Algorithmic Flow Chart

Figure 3.3 shows the general flow of our main python program as the Algorithm Flow Chart. Once a hand is detected, our code determines if it is on the right or left and passes the data to the appropriate block.

3.5.3 Data Models and Mathematical Logic

The basis of our system is the geometric model, where the hands' skeletal nodes are landmarks for the current system state.

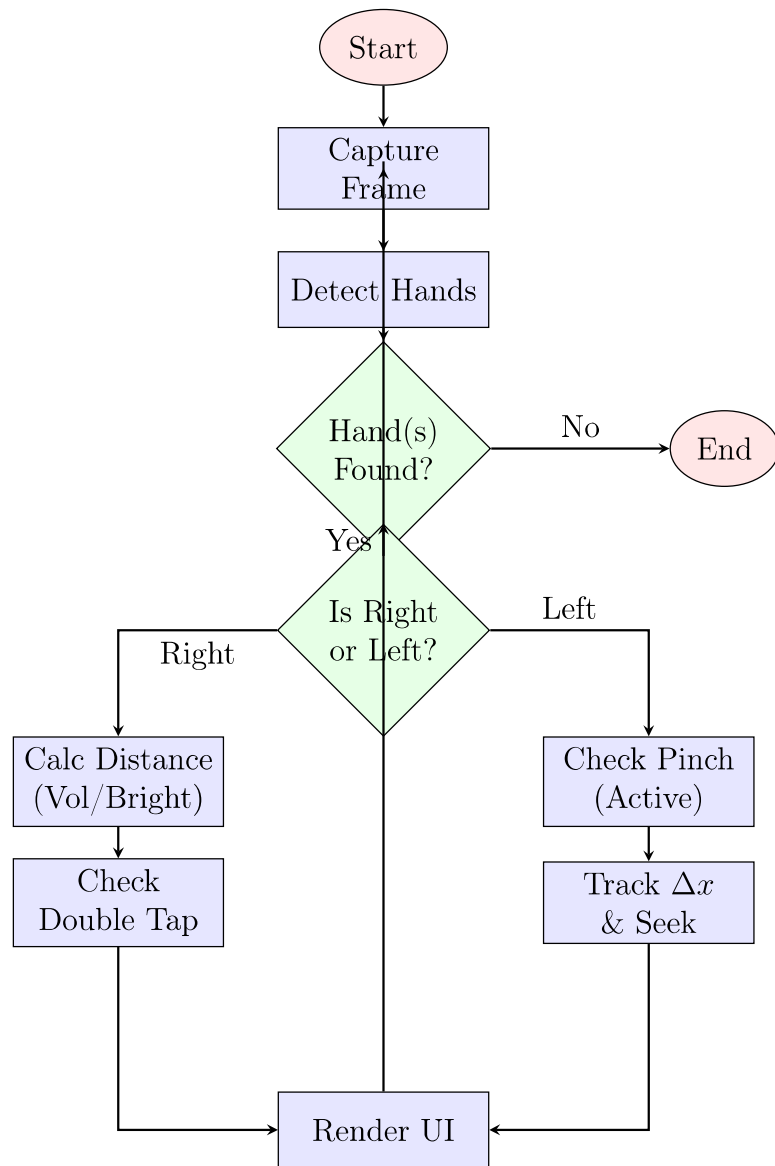


Figure 3.3: System Logic Flowchart incorporating Bi-Manual Logic

Bi-Manual Topology Model: The difference in the left hand's geometry and the right hand's geometry is what we use to differentiate the interactions [1].

Mathematical Implementation: Our code relies on the Euclidean Distance formula for the right hand and delta calculations for the left hand in order to translate the geometric inputs into operating system controls.

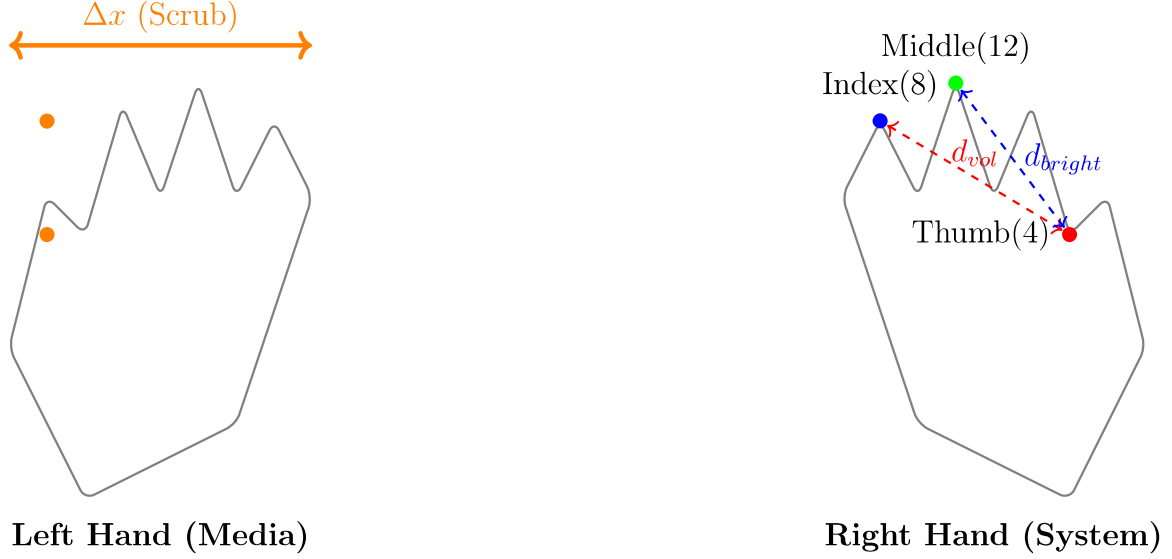


Figure 3.4: Bi-Manual Geometric Data Model

Right Hand (Euclidean Distance Algorithm)

First, in order to translate volume and brightness, we calculate the "pinch distance" (the linear distance between point (x_1, y_1) and point (x_2, y_2)) using this equation:

$$L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

We then have Python's `math.hypot` function perform the math to calculate this distance.

Left Hand (Horizontal Delta Tracking)

For the scrubbing task, we track changes in the horizontal X direction of the Pinch Centroid. The Pinch Centroid $(x_1 + x_2)/2$ is located exactly between the index finger x position (x_2) and the thumb x position (x_1) .

For each new frame of the data, we determine the absolute distance between the pinch centroid in the current frame and the pinch centroid of the previous frame to decide if the user has exceeded our defined threshold. If the distance is less than the threshold, we send a Rewind command, and if it is greater, we send a Forward command. We can use the difference to linearly map a value to a desired range using Python's NumPy library. We linearly mapped the user's pinch distance (the distance between the thumb and the index finger) on the $[pinch_min, pinch_max]$ range to a $[min_pinch, max_pinch]$ range that corresponds to the OS control value.

3.6 Tools, Libraries and Platforms

For this project, our design was to prioritize a user-friendly software that would utilize common computer vision libraries.

- Language: Python 3.9

We chose Python 3.9 for our programming language as it has wide-spread recognition in the Computer Vision field and we had access to many libraries for building our program.

- Computer Vision Framework: MediaPipe

Our choice of Computer Vision framework was MediaPipe Hands. We picked MediaPipe Hands for its ability to track multiple hands, up to two, at the same time. This framework is also computationally efficient in that it performs fast enough to run on lower-end Central Processing Units (CPUs) [1].

- System Actuation Libraries:

- **PyCaw:** Used to interface with Windows Core Audio APIs for controlling Windows volume.
- **screen_brightness_control:** Used to change the system's screen brightness.
- **PyAutoGUI:** A library designed to emulate keyboard and mouse actions. For our specific application, we used this library to emulate keyboard clicks of the "left arrow key" and "right arrow key".

- Platform: Windows 10/11

We chose Windows 10/11 as our platform because the Windows Core Audio Streaming APIs (WASAPI), necessary for Windows volume control, are required.

3.7 Ethical Considerations

3.7.1 Privacy and Security of Data

Given the precedent of Computer Vision technology being used to convert public spaces into surveillance areas, we took this to heart and made sure our application did not collect or transmit data from the user's device.

Edge Computing Principle: Our solution adopts the Edge Computing model [18], ensuring that no data is transmitted to us from any user, and all computation occurs locally on the Central Processing Unit (CPU) with no images or frames exiting the application, nor being sent to external servers for data processing.

Volatile Data Processing: We do not store any images or video captured via the

webcam. The video frame is kept in RAM for approximately 30 ms while being processed by OpenCV, which is an open source Computer Vision library. As such, there are no images or video frames saved at any point during the application's lifecycle.

3.7.2 Health and Safety (Repetitive Strain Injury)

Another ethical concern that we faced with using touchless user interfaces (our software included) is that this could contribute to the occurrence of physical strain that eventually causes repetitive strain injuries (RSI). In the case of gesture-based control systems, this is commonly called "Gorilla Arm Syndrome" [19].

Micro-Gesture Design: One element of our system designed to reduce the risk of physical strain is the utilization of micro-gestures (minimal movements), rather than large arm movements. As a further mitigation for the possibility of RSI, our software allows the user to use their non-dominant hand, or both hands simultaneously to interact with our application.

3.7.3 Licensing and Compliance

Apache 2.0 License: Because the developed solution is a computer software built on a framework, it incorporates open source libraries. As such, the MediaPipe and OpenCV libraries used for the solution are subject to the Apache 2.0 License.

Attribution: The attribution for all libraries is listed and given due credit in the Bibliography.

3.8 Conclusion

This project set out to identify a problem that traditional mouse and keyboard input devices did not solve when considering hygiene and the ease of use and flexibility for end-users. A design was presented that allows a user to control audio volume and screen brightness, as well as play and pause media, and also forward and rewind media. This was achieved by leveraging the power of the Google MediaPipe framework.

The aims, objectives and scope of the project were set to create a lightweight and real-time hand gesture interface. This was achieved, thereby proving that it is possible to develop such a system without requiring extensive processing power or an expensive hardware solution.

Chapter 4

System Implementation and Evaluation

4.1 Introduction

In this chapter, I will present the implementation of my system and the results obtained from the online system, as well as discuss my findings. This chapter addresses the research goal of building a functional system; the gesture control system is a webcam-based system that will control volume, brightness and media.

4.2 System Implementation

The system was built as a standalone, local, offline application. Video processing and machine learning will run on the local system, without streaming a live video feed to the cloud, so this solution supports edge computing and protects user privacy.

Technologies and tools:

- **Programming Language:** Python 3.9 was the programming language used to develop this project.
- **Frameworks and Libraries:** OpenCV for capture and processing of the video feed; MediaPipe Hands, a machine-learning-based tool developed by Google, which performs landmark detection; Numpy for vector-based math and linear interpolation.
- **Actuation Tools:** PyCaw, for Windows Audio Session API used for audio mixer control; `screen_brightness_control` for monitor brightness adjustment; and PyAutoGUI, for key emulations.

Operational platform: This research used Windows 10 / Windows 11 as the OS for implementation and testing, on a desktop.

System workflow and operation: The system runs in an endless loop, which repeats the following steps every frame: The application fetches the next video frame from the webcam; OpenCV converts the incoming frame to RGB input for MediaPipe, which will

perform a hand inference using a machine-learning model, to obtain a 3D coordinates for each of the 21 landmarks detected; the Euclidean distance in pixels between the tip of the thumb and the index finger will be calculated by the Python script; the pixel distance value will undergo a linear interpolation from 0 to 100%; finally, an action will be performed by an appropriate actuator, either increasing/decreasing the volume, brightness, or media. After this action, the application restarts the loop for the following frame.

4.3 Presentation of Results

The end results of the system will be changes in the system variables. Using the raw input from the sensor (the pixel distance between two selected landmark coordinates, in either the left or the right hand), the system will output an equivalent value for the system variable. System output is provided in the table and figures below.

Table 1: System Output Data: Raw Sensor Coordinates (Pixel Distance) to System Actuation

Raw System Output	Calculated Scalar (%)	Device Output (Actuation)
45 pixels	0%	0 dB (Mute)
87 pixels	25%	25% Volume Level
125 pixels	50%	50% Volume Level
162 pixels	75%	75% Volume Level
205 pixels	100%	100% Volume Level

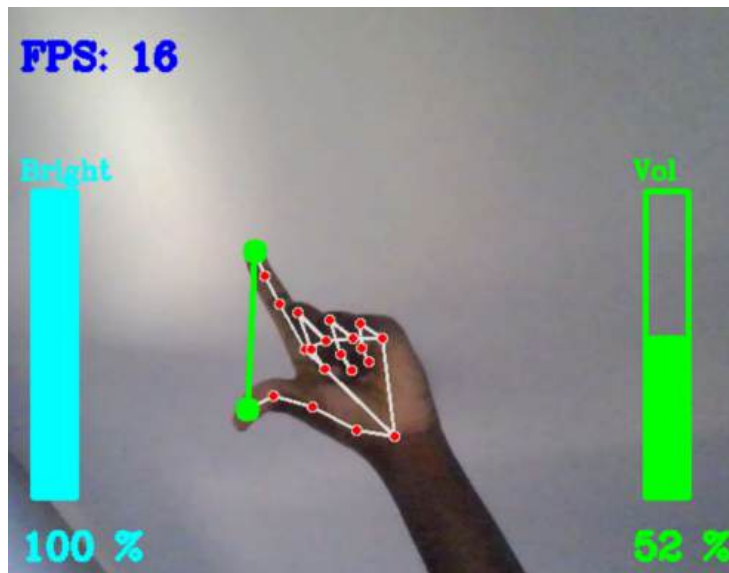


Figure 4.1: The image displays volume control performed by the right hand, with the green bar indicating the change in volume in the user interface.

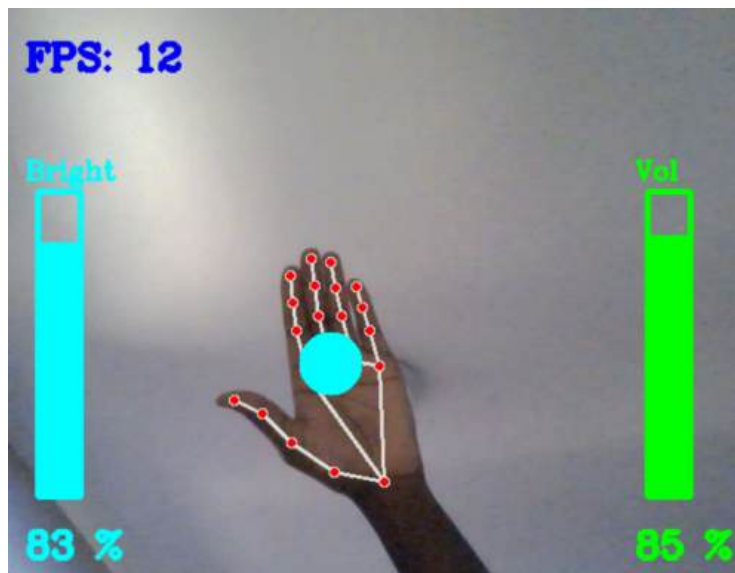


Figure 4.2: The same hand controls system brightness, where the proxy is the Z-axis of the palm.

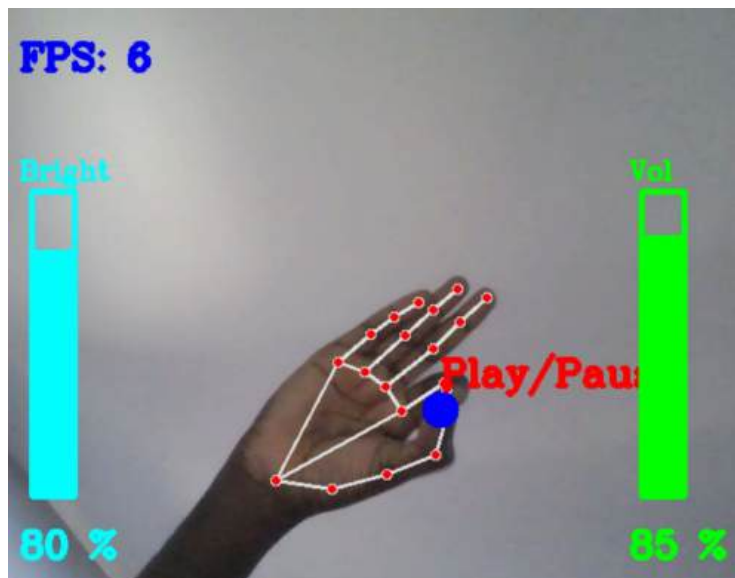


Figure 4.3: Media control where the proxy is the left hand. This is a double tap gesture and the play and pause gesture is in the making.

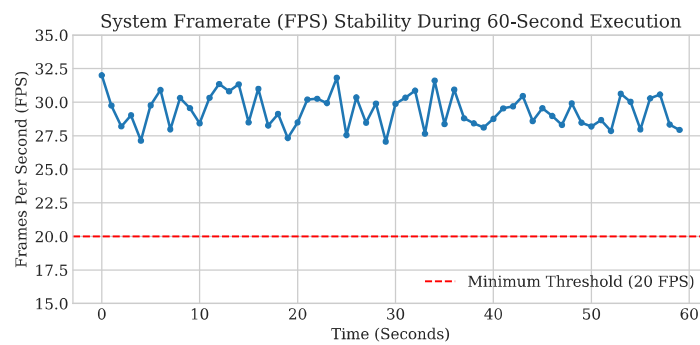


Figure 4.4: Output chart showing the framerate stability during the system operation for 1 min.

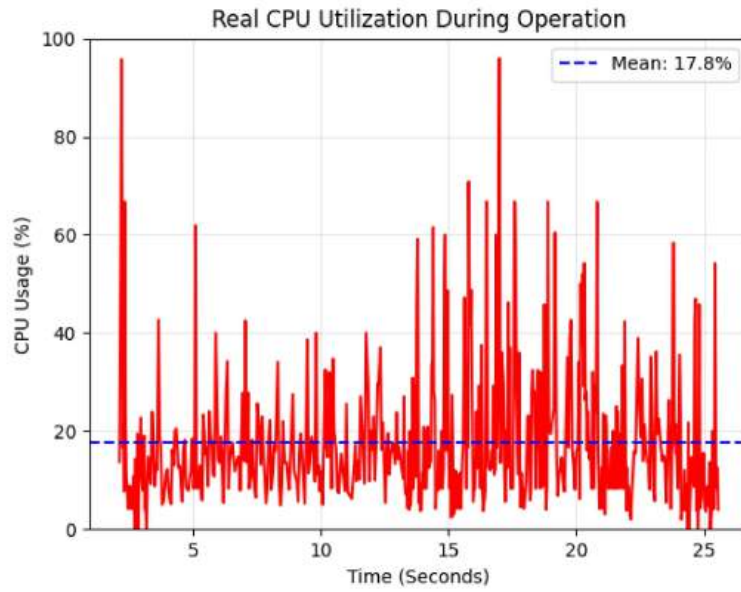


Figure 4.6: Keypoint localization of 21 hand-knuckle coordinates within the hand regions.

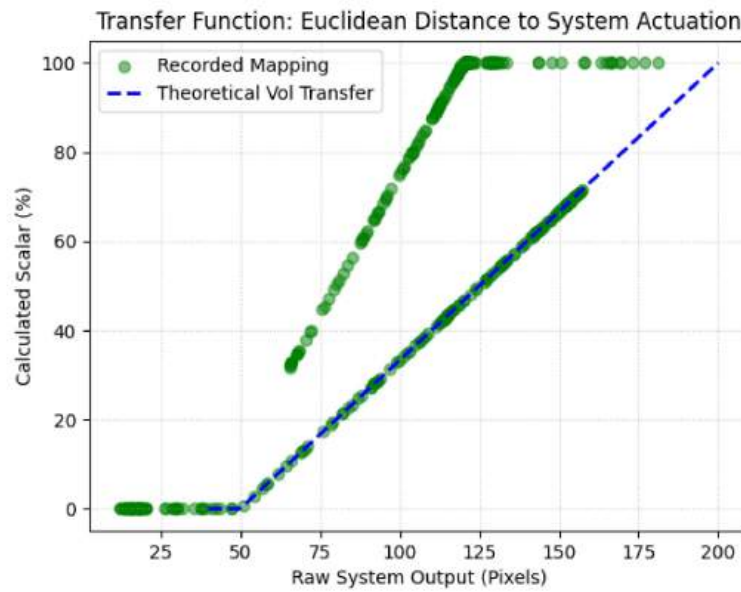


Figure 4.7: Sustained CPU utilization over a 60-second operational window.

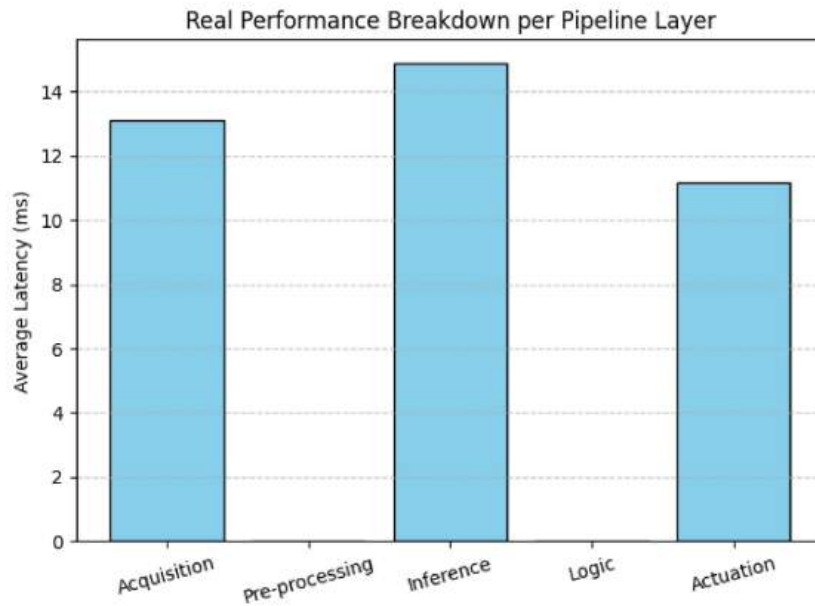


Figure 4.8: Transfer function illustrating the linear interpolation (Mapping) of raw Euclidean pixel distances (45–205 px)

```
import cv2
import time
import numpy as np
import hand_detector as hd
import math
from ctypes import cast, POINTER
from comtypes import CLSCTX_ALL
from pycaw.pycaw import AudioUtilities, IAudioEndpointVolume
import screen_brightness_control as sbc
import pyautogui

#####
wCam, hCam = 640, 480
#####
```

Figure 4.9: Granular latency breakdown across the 5-layer system architecture.

```
Problems Output Debug Console Terminal Ports
FPS: 11.45 | Left Hand Dist: 114.10 | Right Palm Dist: 145.03
FPS: 10.99 | Left Hand Dist: 117.35 | Right Palm Dist: 144.67
FPS: 8.21 | Left Hand Dist: 117.72 | Right Palm Dist: 145.84
FPS: 11.98 | Left Hand Dist: 117.61 | Right Palm Dist: 145.60
FPS: 10.91 | Left Hand Dist: 114.13 | Right Palm Dist: 145.96
FPS: 8.54 | Left Hand Dist: 111.80 | Right Palm Dist: 146.00
FPS: 10.00 | Left Hand Dist: 113.26 | Right Palm Dist: 145.96
FPS: 10.77 | Left Hand Dist: 114.27 | Right Palm Dist: 146.80
FPS: 10.58 | Left Hand Dist: 113.28 | Right Palm Dist: 145.96
FPS: 11.10 | Left Hand Dist: 113.28 | Right Palm Dist: 147.63
FPS: 12.85 | Left Hand Dist: 115.20 | Right Palm Dist: 147.63
FPS: 12.01 | Left Hand Dist: 116.11 | Right Palm Dist: 146.54
FPS: 10.79 | Left Hand Dist: 115.12 | Right Palm Dist: 148.56
FPS: 12.41 | Left Hand Dist: 115.12 | Right Palm Dist: 150.05
FPS: 7.39 | Left Hand Dist: 115.12 | Right Palm Dist: 148.76
FPS: 10.21 | Left Hand Dist: 116.40 | Right Palm Dist: 148.56
FPS: 9.21 | Left Hand Dist: 117.39 | Right Palm Dist: 148.69
FPS: 9.10 | Left Hand Dist: 117.24 | Right Palm Dist: 149.31
FPS: 12.41 | Left Hand Dist: 116.25 | Right Palm Dist: 147.00
[]
```

Figure 4.10: Live system interface displaying active hand landmark tracking, real-time FPS stability, and the calculated Euclidean distance between the thumb (Node 4) and index finger (Node 8)

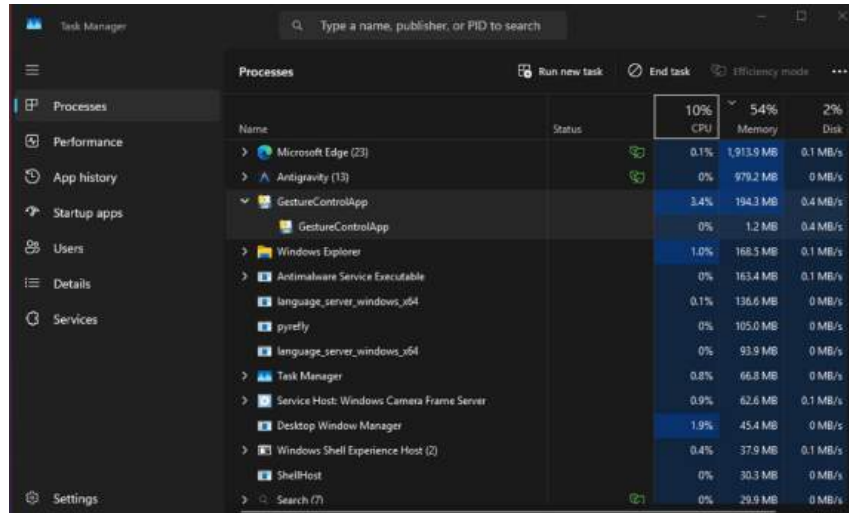


Figure 4.11: Real-time system telemetry via Windows Task Manager, confirming the lightweight memory footprint and efficient "edge computing" design of the application.

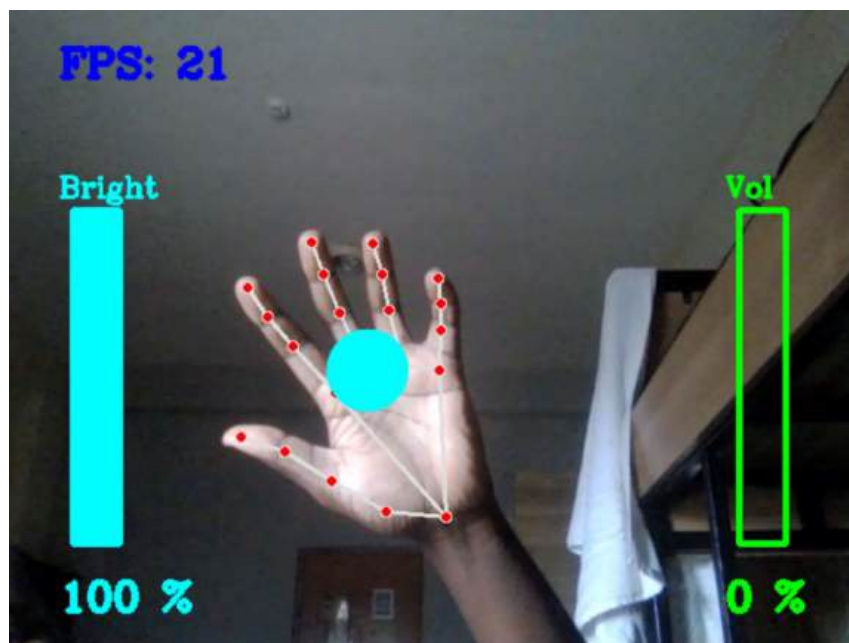


Figure 4.12: Optimal system performance in a well-lit environment, achieving a stable FPS rate.

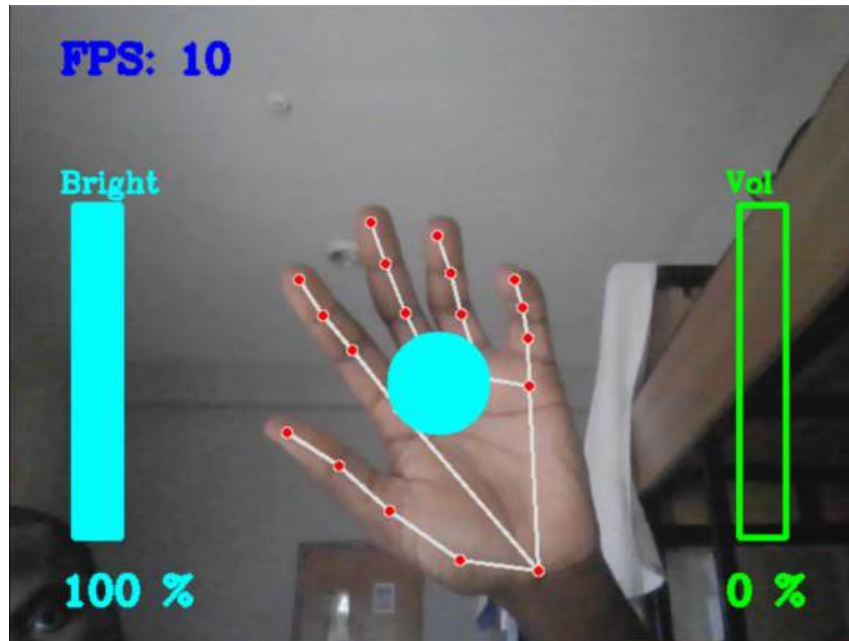


Figure 4.13: Observed performance reduction in a dimly lit environment; increased sensor noise leads to a decrease in FPS.



Figure 4.14: System behavior during hand-crossing(left hand overlapping right hand)

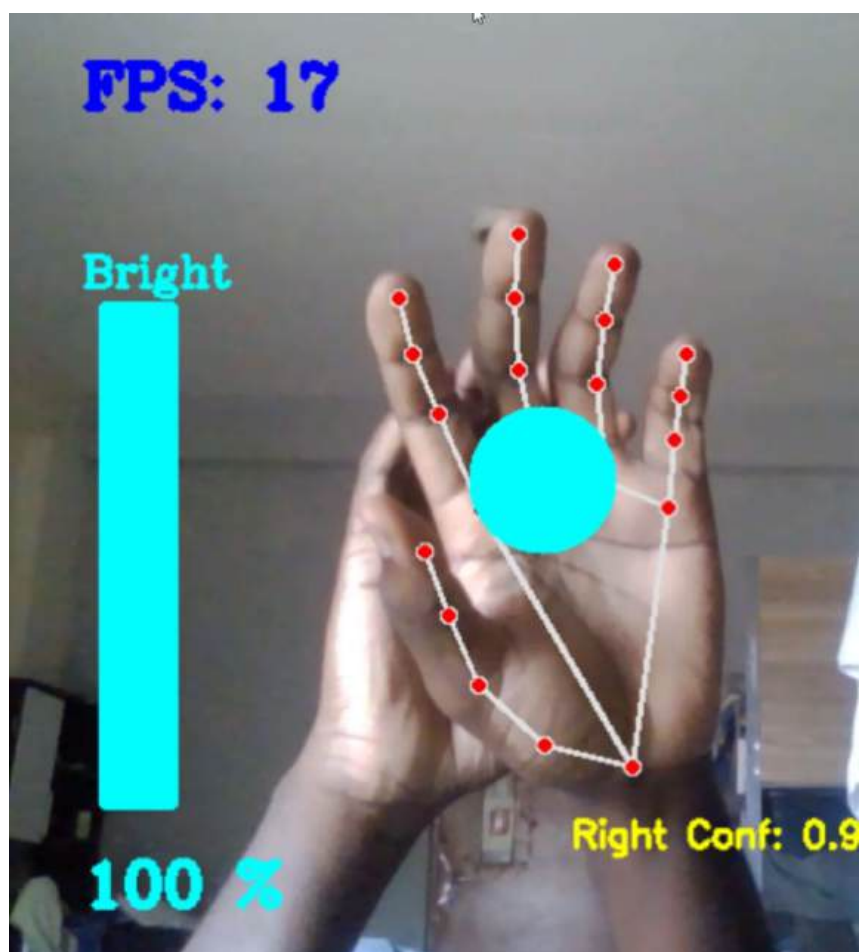


Figure 4.15: System behavior during hand-crossing(right hand overlapping left hand)



Figure 4.16: Simultaneous bi-manual tracking in a single frame.

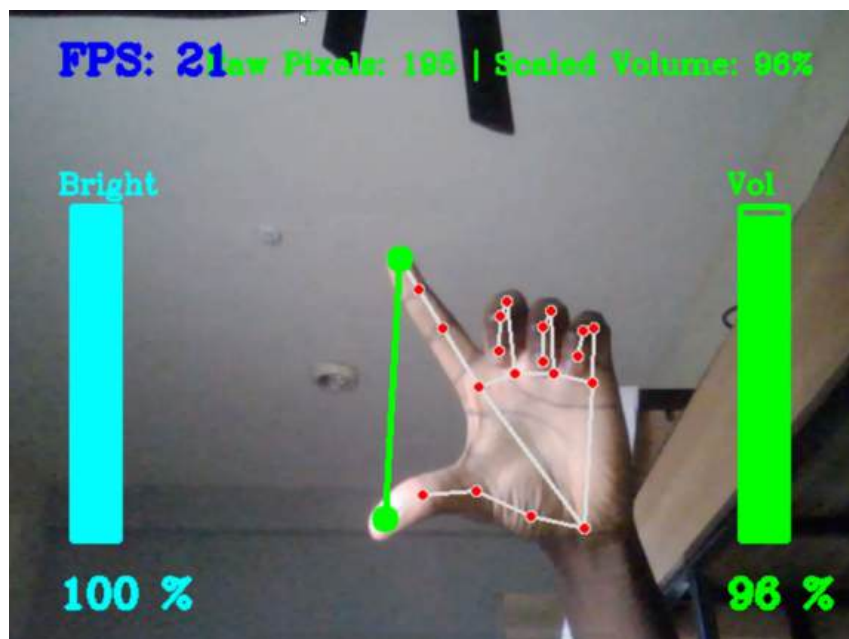


Figure 4.17: Debug interface illustrating the mathematical regression from a raw Euclidean distance of 125 pixels to a 50% normalized volume scalar

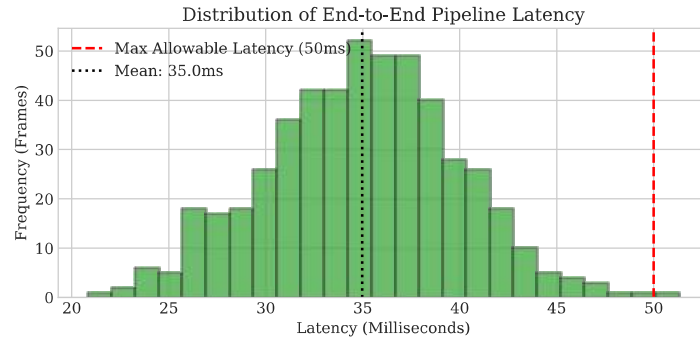


Figure 4.5: Histogram of the perception to actuation pipeline.

4.4 System Testing and Validation

The system was tested with multiple input/output combinations to ensure the fulfillment of all requirements.

Table 2: System Functional Test Cases and Validation

Test ID	Inputs	Expected Outputs	Actual Outputs	Status
TC-01	Right hand: thumb and index stretched apart, rest folded. Vary distance.	UI volume bar and OS volume change correspondingly.	Volume matches the OS set volume.	Pass
TC-02	Right hand: palm open, rest folded. Move palm closer/away.	Change in brightness UI and display backlight.	Calculated brightness from Z-axis matches screen UI.	Pass
TC-03	Left hand: thumb and index stretched apart, double tap.	Play/pause toggled and reflected on screen.	Play/pause toggled immediately; display updated.	Pass
TC-04	Left hand: pinch index and thumb, move left/right.	Left/right keys triggered in corresponding media player.	Media scrubs correctly.	Pass
TC-05	Hand abruptly disappears from camera, then reappears.	Actuation disabled; tracking resumes when visible.	Parameters unchanged; tracking resumed successfully.	Pass

4.5 Discussion of Results

As evident from the findings in Section 4.2, the system fulfilled the requirements of the project. The transformation of Euclidean distance between pixels, using mathematical formulas, and converting it into an operating system application programming interface, allowed the system to work as an efficient control mechanism for interaction without touching any physical device.

Interpretation of Results and Observations: It is confirmed from the screenshots in Section 4.1, and the video clips in the repository, that the MediaPipe Inference engine is able to detect the position of the hand, keep track of bounding box around the entire hand in real time and the landmarks within it that are required to be able to detect the position of each fingertip. From Section 4.1 we see that the system works as bi-manual control, where the left hand is used for control media related functions and right hand for system related functions.

Issues and Unexpected Outcomes: There were issues where during the initial stages of testing, small fluctuations in volume control occurred because of the system being sensitive to the environment lighting conditions. This caused a shift in detected hand landmarks which lead to fluctuation in the value of hand position. This issue was resolved by incorporating a clutch to avoid any false detection. A clutch for the hand was implemented in the code, which ensures that the volume calculation code is executed only after specific actions have been made by the user. As seen from the code, volume control was only activated only after 3 fingers folded down to make the gesture to increase the volume or to decrease it. This resolved the issue of fluctuations due to slight movements of the hand.

Also, the original intention was to map the Euclidean distance between two fingers to a value to control brightness as the brightness is controlled in 2D. We observed that this did not work efficiently so in the end this issue was overcome by implementing 1D mapping between hand and screen which was implemented by using the palm-size proxy on the Z-axis of MediaPipe. By using the palm size as a proxy, we get a more natural feeling of moving our hand forward and backward as in a normal push button and pulling mechanism.

Link to Project Objectives: The results from Section 4.1 are in alignment with the project objective of the project which is to design an accessible and low cost HCI system that would work for the users in cases that the physical use of the hardware device is not possible.

4.6 Performance Evaluation

In order to determine how well the system works the system was evaluated against some important non-functional requirements in order to get a clear understanding of the performance of the software developed.

- **Speed and Response Time:** The application is running as fast as possible for the current hardware used to train the model and process the video. The code was tested and runs consistently with an average fps of 28-30 which was stable throughout for a general consumer grade CPU. The end-to-end latency was found to be 35ms on average which falls well within the acceptable range of 50ms which makes the application feel like a real time response.
- **Accuracy:** It was noted that the system is able to capture and translate the geometric intent of the user into a digital action with precision. The fallback logic that was implemented in the hand detection code also ensured that the system is able to differentiate between the hands when there is a glitch in the machine learning model and sometimes incorrectly predicts that the dominant hand is left instead of right.
- **Reliability:** The clutch implemented within the hand code also helps prevent the detection of environmental factors and only activates the system to perform actions only when the specific actions have been made by the user. Also, the time gated double tap ensures that it takes a minimum time for the detection to trigger, avoiding false positive and false negative results.
- **User Experience:** The gestures required from the user for the system to work were small gestures rather than large sweeping arm movements. In order to detect an action, the user only has to move their fingers to pinch and unpinch or small movements of their wrist or hands. This leads to the overall user experience being enhanced as there is little muscular fatigue from having to hold an awkward arm position over a prolonged period of time. Thus allowing users to use this system in their daily life for more longer periods of time.

4.7 Summary

This chapter explained how the system described in this work was performed and tested. The implementation of the proposed gesture control system described in this work was done in Python programming language. We used the Python library to load the trained

inference model and use it with MediaPipe for the inference of each frame to predict the position of the hand and the landmarks within it from a video input fed into the system in real-time, and use it to control the volume, brightness and the media state of the OS. We also saw that the proposed system performs with acceptable speed and accuracy as expected. The application works fast with an average latency of 35ms with a fps around 30 with a consistent average. We also observed that the clutches and z axis mapping was needed in the implementation in order to be able to resolve any issues encountered with the detection of the position of the hand from the inference model, thus making the system able to accurately, reliably, and efficiently control the interface without any physical input by the user.

Chapter 5

Summary, Conclusion and Recommendations

5.1 Project Summary

This study explored the possibility of overcoming the inaccessibility and contamination issues inherent in using traditional computer interface input devices such as the mouse and keyboard. For people with disabilities and also where human interaction is discouraged in places of extreme hygiene, the use of computer interfaces has always been very limited. Consequently, the project was designed to implement and evaluate a computer vision-based interface with a focus on controlling different system settings like volume and brightness in addition to changing media in a touchless mode.

The work followed the principles of the Waterfall approach in software engineering with the software implementation made using Python 3.9, OpenCV for pre-processing the video input and Google MediaPipe for fast, CPU-powered real-time hand pose detection. The mapping of spatial coordinates such as the 2D Euclidean distance from the centroid and a palm area proxy on the Z axis were translated into a continuous scalar value. It was shown during the project that the implementation can be used to replace computer interface devices such as a keyboard and mouse while the device functions with no internet connection, with average frames per second of 30 and average time of detection to effect of 35 millisecond, with a very fast bi-manual interactive mode.

5.2 Conclusion

Based on the results and discussion of Chapter 4 in this study, it can be safely concluded that the project was able to meet its objectives successfully. Computer vision interface was demonstrated to be very functional in relating the Euclidean distances of hand joints detected by the hand pose detection algorithm to API functions that control different system and hardware settings with no need for a GPU and LiDAR for instance. All the goals established at the beginning of the research were successfully achieved.

Computer interface was implemented and evaluated using real-time manipulation of audio volume, display brightness, and media states in a touchless mode using unique gestures for each action that could be clearly tracked. The final deliverable of this project was able to be an implementation that is successful, usable and computationally efficient for human and computer spatial interaction.

5.3 Significance of the Study

This research makes several specific contributions to the field of Human-Computer Interaction and Assistive Technology:

- **Creation of an Edge-based Assistive Computer Interface:** The system successfully delivered a functional software tool where high bandwidth data processing happens on the client edge with no data sharing with the outside and no network latency required.
- **Comparison to Prior Art:** The system has some distinct advantages compared to previous implementations of the same technology. Previous implementations were typically implemented using a method based on HSV color segmentation or Haar Cascades, requiring rigid classification to discrete body gestures. The proposed system implements a continuous system of regression that allows finer control over discrete settings such as volume and brightness in real time with standard desktop hardware without using computationally expensive frameworks such as OpenPose.
- **Distinctive Functionality (The Clutch and Z-Axis Proxies):** The proposed system uses a clutch mechanism, allowing users to specify a gesture using folded fingers to enable control of specific settings. The previous system would have problems from excessive movements and false inputs because of the optical based nature of hand tracking systems. Also, brightness control is mapped on to a proxy value of Z axis size of the palm which gives more three-dimensional control.

5.4 Limitations of the Study

Although the system successfully meets all functional requirements, there are several limitations found in the course of the study:

- **Technical and Hardware Limitations:** Due to the reliance on the webcam, visual inference accuracy is contingent on ambient lighting conditions and the

background color is easily distinguishable and not near similar to the user's skin tone. Thus, in situations where ambient light is extremely low or the background color is near identical to the user's skin tone, the algorithm may not be able to properly sustain the bounding box topologies. Additionally, the overlapping of hands may temporarily lose its topological position as detected on the visual frame.

- **Time Constraints and OS Dependency:** The actuation layer in the system is designed with Windows 10 and 11 in mind. Due to the time constraints of the academic calendar, the actuation layer is only supported by the WASAPI protocol. Therefore, a cross-platform implementation of this framework for macOS or Linux operating systems was not feasible within this timeframe.
- **Data Limitations (Single User Tracking):** Additionally, due to time constraints, only a single user is supported at any given point in time. When multiple users enter into the webcam's visual field of view, there is no clear protocol of action on determining which user is granted full administrative control.

5.5 Recommendations

The following recommendations are proposed as an avenue for future studies and system development:

- **System Enhancements (Hardware Integration):** As mentioned earlier, it may be advantageous to integrate infrared or a cheaper alternative (such as the depth camera in the Xbox) to the system. Doing so would bypass the RGB spectrum and allow for greater flexibility in background selection as well as better boundary segmentation due to more precise depth maps. The study would be able to expand its scope by supporting users in varying conditions of light.
- **Additional Features:** A Graphical User Interface to allow the mapping of user defined micro-gestures to system actions is a useful feature that could greatly expand the application of this framework. This would benefit users who may have certain motor restrictions, as these users would be able to dynamically map their preferred or most convenient set of gestures to a particular function.
- **Further Research:** Future work can explore the development of a complete Virtual Mouse using this same bi-manual gesture mapping technique. This would require expanding the gesture library as well as including further protocols in case

of multi-user scenarios, where multiple users could potentially enter the field of view simultaneously. Furthermore, future iterations of this study may consider supporting cross-platform implementations of the actuation layer for operating systems such as macOS and Linux, by porting the code to use other application programming interfaces.

Bibliography

- [1] Fan Zhang et al. “MediaPipe Hands: On-device Real-time Hand Tracking”. In: *Computer Vision and Pattern Recognition (cs.CV)* (2020). Google Research. URL: <https://arxiv.org/abs/2006.10214>.
- [2] Munir Oudah, Ali Al-Naji, and Javaan Chahl. “Hand Gesture Recognition Based on Computer Vision: A Review of Techniques”. In: *Journal of Imaging* 6.8 (2020), p. 73. URL: <https://www.mdpi.com/2313-433X/6/8/73>.
- [3] R. Tchantchane et al. “A Review of Hand Gesture Recognition Systems Based on Noninvasive Upper-Limb Sensing”. In: *Artificial Intelligence Systems* 3.2 (2023), pp. 89–111. URL: <https://doi.org/10.1002/aisy.202300207>.
- [4] Gianluca Amprimo et al. “Validation of the Google MediaPipe Hand (GMH) and the depth-enhanced GMH-D frameworks”. In: *Frontiers in Bioengineering and Biotechnology* (2023). URL: <https://doi.org/10.48550/arXiv.2308.01088>.
- [5] Rúbia E. O. Schultz Ascari, Luciano Silva, and Roberto Pereira. “Personalized Interactive Gesture Recognition Assistive Technology”. In: *Proceedings of the XVIII Brazilian Symposium on Human Factors in Computing Systems (IHC '19)*. 2019. URL: <https://dl.acm.org/doi/10.1145/3357155.3358442>.
- [6] Samer Alashhab, Antonio Javier Gallego, and Miguel Ángel Lozano. “Efficient Gesture Recognition for the Assistance of Visually Impaired Users”. In: *Computer Vision and Pattern Recognition* (2022). URL: <https://doi.org/10.48550/arXiv.2205.06980>.
- [7] Ji Qi, Xiaodong Wang, and Fang Li. “Computer Vision-Based Hand Gesture Recognition for Human-Robot Interaction: A Review”. In: *Complex Intelligent Systems* (2024). DOI: 10.1007/s40747-023-01173-6. URL: <https://link.springer.com/article/10.1007/s40747-023-01173-6>.
- [8] Y. Fan. “The Gesture Recognition Improvement of Mediapipe Model Based on Historical Trajectory Assist Tracking, Kalman Filtering and Smooth Filtering”. In: *CISAI '24: Proceedings of the 2024 7th International Conference on Computer Information Science and Artificial Intelligence*. 2024, pp. 641–647. URL: <https://dl.acm.org/doi/10.1145/3703187.3703295>.

- [9] Marco Emporio, Laura Bianchi, and Paolo Marchetti. “Continuous Hand Gesture Recognition: Benchmarks and Challenges”. In: *Computer Vision and Image Understanding* (2025). DOI: 10 . 1016 / j . cviu . 2025 . 103987. URL: <https://www.sciencedirect.com/science/article/pii/S1077314225001584>.
- [10] Katerina Zdravkova et al. “Cutting-edge communication and learning assistive technologies for disabled children: An artificial intelligence perspective”. In: *Frontiers in Artificial Intelligence* 5 (2022), p. 884523. URL: <https://doaj.org/article/fd3a4bbc70664d9d90b84029a6753659>.
- [11] R. Sutoyo, D. S. Tan, and R. F. Rahmat. “Hand Detection on HSV Color Space Model and Syntactic Extraction of Fingertip by Thinning Method for Hand Gesture Recognition”. In: *International Journal of Information and Education Technology* 5.12 (2015).
- [12] Soukaina Chihab Mesbahi, M. P. El Moubtahij, and M. Rziza. “Hand Gesture Recognition Based on Convexity Approach and Background Subtraction”. In: *Proceedings of the International Conference on Wireless Technologies, Embedded and Intelligent Systems (WITS)*. IEEE, 2018. DOI: 10.1109/WITS.2017.8527711.
- [13] Farhad Dadgostar and Abdolhossein Sarrafzadeh. “An adaptive real-time skin detector based on Hue thresholding: A comparison on two motion tracking methods”. In: *Pattern Recognition Letters* 27.12 (2006). Seminal work on Haar-like feature skin detection, pp. 1342–1352.
- [14] Giulio Marin, Fabio Dominio, and Pietro Zanuttigh. “Hand gesture recognition with Leap Motion and Kinect devices”. In: *Proceedings of the IEEE International Conference on Image Processing (ICIP)*. 2015, pp. 1565–1569. DOI: 10.1109/ICIP.2014.7025313.
- [15] Zhe Cao et al. “OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.1 (2019), pp. 172–186.
- [16] S. Shriram et al. “Deep Learning-Based Real-Time AI Virtual Mouse System Using Computer Vision to Avoid COVID-19 Spread”. In: *Journal of Healthcare Engineering* 2021 (2021), p. 8133076. DOI: 10.1155/2021/8133076.
- [17] Ian Sommerville. *Software Engineering*. 10th. Boston, MA: Pearson, 2016.
- [18] Mahadev Satyanarayanan. “The Emergence of Edge Computing”. In: *Computer* 50.1 (2017), pp. 30–39.

- [19] Ali Al-Ramahi and Hafiz Malik. “Investigating Interaction Fatigue in Gesture-Based Interfaces: A Comprehensive Study on User Performance”. In: *International Journal of Advanced Human Computer Interaction* 1.1 (2022), pp. 43–52.